

Aim:

To set up and launch the Azure DevOps environment by creating an organization through Azure portal.

Installation:

1. Open your web browser and go to the Azure website. Sign in using your Microsoft account credentials. If you don't have a Microsoft account, you can create one.

2. Azure home page.

3. Open DevOps environment in the Azure platform by typing Azure DevOps Organizations in the search bar.

4. Click on the My Azure DevOps Organization link and create an organization and you should be taken to the Azure DevOps Organization Home page.

Result:

Successfully accessed the Azure DevOps environment and create a new organization through the Azure portal.

Exp: 2

Azure DevOps Project Setup And User Story Management

(Create Epic, Features, User Stories Task)

Aim:

To set up an Azure DevOps project for efficient collaboration and agile work management.

1. Create an Azure Account.

2. Create the first Project in your Organization:

a. After the organization is set up, you'll need to create your first project. This is when you begin to manage code, pipelines, work items, and more.

b. On the organization's Home page, click on the New Project button.

c. Enter the project name, description, and visibility options:

Name: Choose a name for the project (e.g., LMS).

Description: Optionally, add a description to provide more context about the project.

Visibility: Choose whether you want the project to be Private or Public.

d. Once you've filled out the details, click Create to set up your first project.

3. Once logged in, ensure you are in the correct organization. If you're part of multiple organizations, you can switch between them from

the top left corner. Click on the Organization name, and you should be taken to the Azure DevOps Organization Home page.

4. Project Dashboard.

5. To manage user stories:

a. From the left-hand navigation menu, click on Boards. This will take you to the main Boards page, where you can manage work items, backlogs, and sprints.

b. On the work items page, you'll see the option to Add a work item at the top. Alternatively, you can find a + button or Add New Work Item depending on the view you're in. From the Add a work item dropdown, select User Story. This will open a form to enter details for the new User Story.

Result:

Successfully created an Azure DevOps project with user story management and agile workflow setup.

Setting Up Epics, Features and User Stories for Project Planning

Aim:

To learn about how to create epics, user story, features, backlogs for your assigned project.

Create Epic, Features, User Stories, Task:

1. Fill in Epics
2. Fill in Features.
3. Fill in User Story Details

Result:

Thus the creation of epics, features, user story and task has been completed.

Exp: 4

Sprint Planning

(Create Epic, Feature, User Stories, Task)

Aim:

To assign user story to specific sprint for the Music Playlist Batch Creator Project.

Sprint Planning:

Sprint 1

Sprint 2

Sprint 3

Sprint 4

Member 1: 2

Member 2: 2

Member 3: 8

Member 4: 2 (Total)

Result:

The sprints are created for the Music Playlist Batch Creator Project.



19/2/20

Ex: 5 Poker Estimation

Aim:

Create poker estimation for the user stories, main
playkit batch creation project.

Poker Estimation:

- In Azure microsof account click your main
engonig project.
- Click new and create a user story and add
story pts.
- Each member in the team should add their story points
in planning column. Eg:

member 1 : 3

member 2 : 5

member 3 : 8

(tester) QA : 8

Finally Every team member go with the story
point 5. According to the poker estimation.

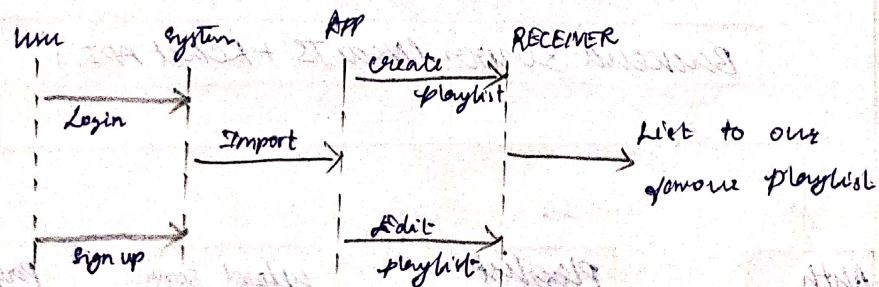
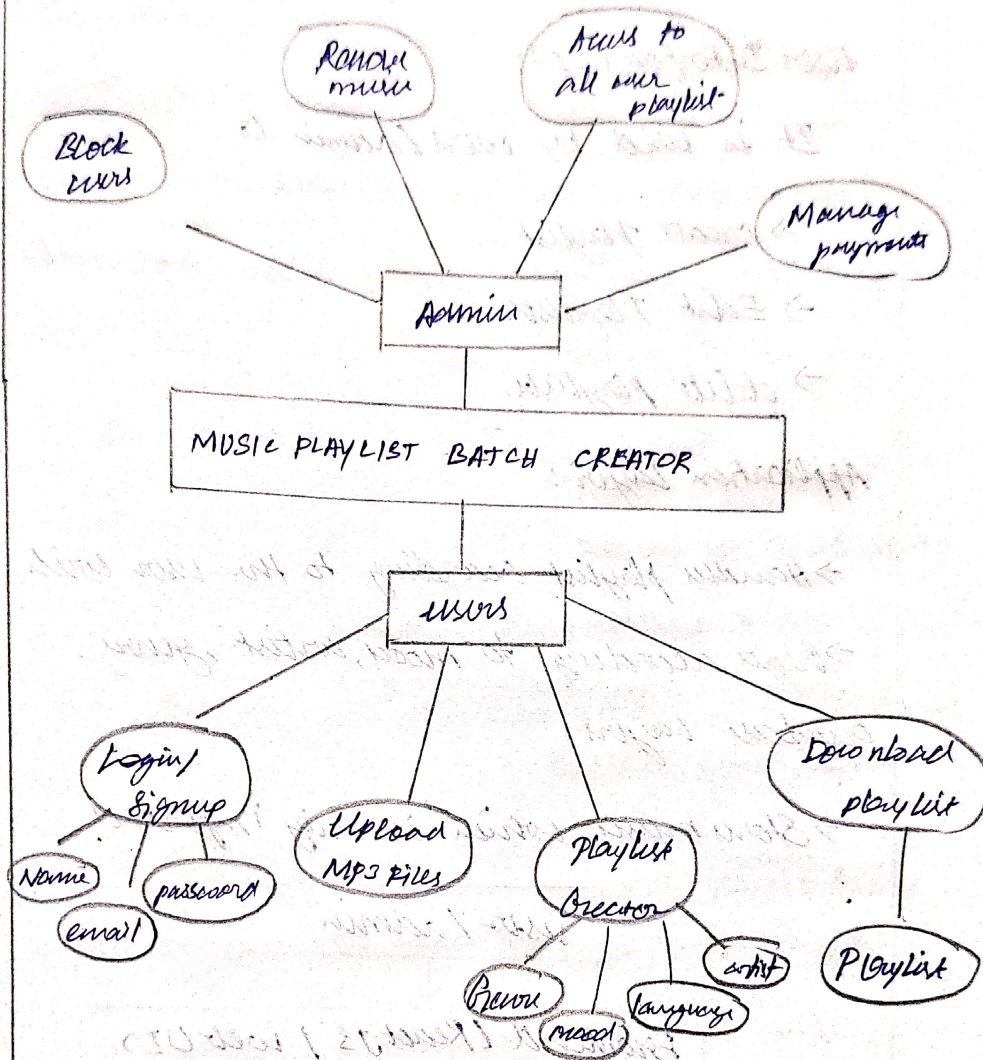
Result:

The Estimation of story points is created for
the project using poker estimation.

26/2/26

Ex:6 Designing class and sequence diagrams for project Architecture.

Aim: To design a class diagram and sequence diagram.



Result:

This is the class diagram and sequence diagram is drawn.

5/3/26

Ex: 7 Designing Architecture

Aim:

To design an architecture diagram and ER diagram for Music Playlist web browser.

User Interface (UI):

It is used by users/admins to

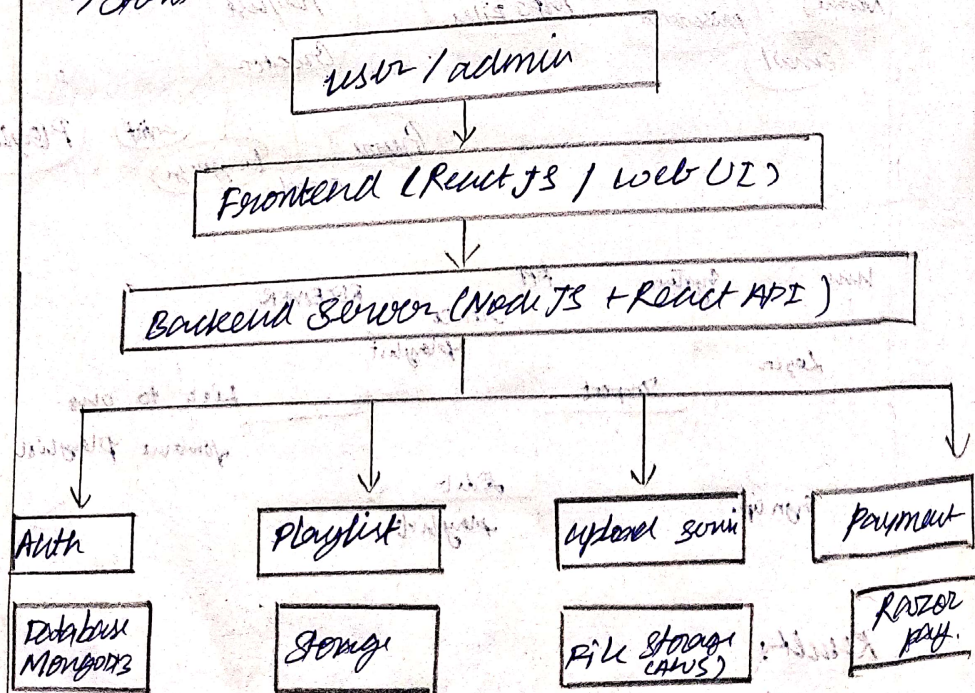
- create playlist.
- Edit playlists.
- delete playlists.

Application Layer:

- handles playlist according to the user wish.
- create according to mood, artist, genre.

Database Layer:

- stores music, which is only imported.



Result:

The architecture diagram is drawn successfully.

Aim:

To design test plans and test cases for the Music Playlist Batch Creator project.

Description:

Test plans were created for key features like login, playlist creation, editing and metadata. Both happy path and error scenarios were tested to ensure reliability.

Sample Test Case:

Login success: User enters valid credentials → redirected to dashboard.

Login failure: Wrong password → error message shown

Create Playlist: Playlist created successfully.

Empty Playlist Name: Error displayed.

View Playlist: All playlist shown.

Offline Mode: Playlist loading fails with error.

Result:

Test cases were successfully created and executed for major user stories ensuring proper functionality.

Ex: 9 Load Testing and Pipelines

Aim:

To perform load testing and implement CI/CD pipeline for the project.

Description:

- * Load testing was done using Azure to check performance under high traffic.
- * CI/CD pipeline was created using Azure DevOps connected with GitHub.

Pipeline Tasks

- * Code checkout.
- * Python environment setup.
- * Install dependencies.
- * Run test script.

Outcome:

- * Application handled multiple requests efficiently.
- * Pipeline automated build & testing process.

* Result:

Load testing and pipeline setup were completed successfully for performance and automation.

Aim:

To organize the project structure and maintain proper naming conventions.

Project Structure Example:

- /project → main application code
- /templates → HTML files.
- /static → CSS, JS.
- /tests → test cases.
- requirements.txt → dependencies.
- app.py → main file.

Naming conventions:

→ Files: lowercase with underscores.

(playlist-manager.py)

→ Variables: meaningful names

(user-playlist)

→ Functions: action-based

(create-playlist())

Result:

Project structure was organised clearly, improving reliability and maintainability.